- Programación Orienada a Objetos (CAS)
  - 1. Métodos y atributos estáticos
    - Actividad 8: Uso de métodos y atributos estáticos
  - 2. El patrón de diseño Singleton
    - Actividad 9: La clase biblioteca
    - Actividad 10: Pruebas unitarias con singleton
  - 3. Javadoc
    - Actividad 11: Documenta el proyecto Biblioteca
  - 4. Diagramas en PlantUML y markdown
    - Actividad 12: Diagramas en Markdown
- Programació Orientada a Objectes (VAL)
  - 1. Mètodes i atributs estàtics
    - Activitat 8: Ús de mètodes i atributs estàtics
  - 2. El patró de disseny Singleton
    - Activitat 9: La classe biblioteca
    - Activitat 10: Proves unitàries amb singleton
  - 3. Javadoc
    - Activitat 11: Documenta el projecte Biblioteca
  - 4. Diagrames en PlantUML i markdown
    - Activitat 12: Diagrames en Markdown

# Programación Orienada a Objetos (CAS)

## 1. Métodos y atributos estáticos

Existe una palabra reservada que hemos estado utilizando hasta ahora sin saber muy bien qué era: `static`. Los métodos y atributos estáticos son aquellos que no pertenecen al objeto, sino a la clase.

Esto quiere decir que una vez que el programa es ejecutado, son siempre accesibles independientemente de si hay o no instancias de la clase que los contiene.

En las primeras etapas del curso hemos usado los elementos estáticos para simular el paradigma de programación procedural en Java, es decir, la resolución de problemas mediante funciones/procedimientos y no mediante objetos.

Si declaramos una atributo como estático, se convierte en una especie de variable con ámbito dentro de toda la clase y a la cual podemos acceder desde fuera dependiendo de su visibilidad. Lo mismo sucede con los métodos.

Una cosa que hay que tener en cuenta es que no se pueden usar atributos de la clase dentro de métodos estáticos (porque, en realidad, no tiene sentido ya que no hay instancia de la clase), pero sí podemos usar atributos estáticos dentro de métodos del objeto.

En cierto modo, podemos usar las clases como contenedores de métodos estáticos, que se comportan como librerías de funciones. Tal es el caso de la clase `Math`, que incluye métodos estáticos para operaciones matemáticas complejas.

De esta manera, podemos emplear los atributos y métodos estáticos como herramientas útiles para emplear dentro o fuera de la clase.

### *Actividad 8: Uso de métodos y atributos estáticos*

Crea una variable estática dentro de la clase libro que sirva para contar el número de libros (instancias) que hay. Empléala para darles a cada libro un identificador único. Para ello, añade un atributo privado `id` de tipo entero. El `id` no debe pasarse como atributo al constructor ni debe poder ser modificado con un setter.

Después, crea un método estático dentro de la clase Estantería que reciba como parámetro una estantería y devuelva un array de string. El contenido del array de string es cada uno de los parámetros, incluido el `id`, separado por el caracter `;`. Puedes rodear cada parámetro entre ` ` si dentro de alguno de los campos hubiera un `;`. Cada libro ocupa un espacio distinto del array. A este formato de texto se le conoce como CSV (Comma Separated Values) y es muy empleado en la codificación de datos.

---

## 2. El patrón de diseño Singleton

Uno de los usos más particulares de los atributos y métodos estáticos es mediante el patrón de diseño Singleton. Este patrón garantiza que solo haya una instancia de una clase haciendo que el constructor sea privado.

Para que funcione, dentro de la clase debemos añadir un atributo privado y estático de la propia clase, así como un getter para ese atributo. En el getter, si la instancia no está inicializada, se llama al constructor. Después, en cualquier caso, se devuelve la instancia.

De esta forma, si no hay instancia, se crea una y, si la hay, obtienes la que ya había.

```
public class Singleton {
    // Instancia privada y estática de la clase
    private static Singleton instancia;

    // Constructor privado para evitar la instanciación desde fuera
    private Singleton() { }

    // Método público y estático que proporciona acceso a la única
instancia
    public static Singleton getInstancia() {
        if (instancia == null) {
            instancia = new Singleton(); // Crear instancia si no existe
        }
        return instancia; // Retornar la instancia única
    }
}
```

Aunque el patrón singleton es muy útil para almacenar datos globales y de configuración, no está exento de problemas:

- **Dificultad para el testeo y el mantenimiento:** La naturaleza global de la instancia puede complicar las pruebas unitarias, ya que introduce un acoplamiento fuerte. El acoplamiento ocurre cuando un módulo o clase es altamente dependiente de otro, es decir, necesita conocer detalles específicos de la implementación del otro para funcionar y, por lo tanto, es más complejo de mantener.
- **Problemas de concurrencia:** En entornos multihilo, puede haber problemas si no se maneja adecuadamente el acceso a la instancia. Esto se soluciona con los mecanismos propios de la programación concurrente, pero añaden complejidad a la ejecución del programa.

## *Actividad 9: La clase biblioteca*

La clase `Biblioteca` contiene diez estanterías, numeradas del 1 al 10. Añade este atributo a la clase `Estanteria`. Solo existe una biblioteca. La biblioteca permite hacer las siguientes cosas:

- Añadir libros nuevos. Se añaden a la primera estantería disponible. Devuelve falso si la biblioteca está llena.

- Añadir libros nuevos a estanterías concretas. Devuelve falso si la estantería no tiene hueco.
- Eliminar un libro. Se le pasa el isbn del libro y, si está, devuelve verdadero y lo elimina. Si no está, devuelve falso.
- Cambiar un libro de estantería. Se le pasa el isbn del libro y el número de la estantería. Devuelve verdadero si se puede cambiar o si ya estuviera en la estantería. Devuelve falso si la estantería nueva está llena o si el libro no existe en la biblioteca.
- Mejora el método anterior con un enum que indique lo que ha pasado en cada caso.
- Intercambia dos libros de estantería. Recibe como parámetro el isbn de los dos libros y devuelve verdadero si el cambio es posible y falso si no lo es (por ejemplo, porque un libro no existe en la librería).
- Un método que, dado el isbn o el título de un libro, te devuelva la estantería en la que se encuentra.
- Un método que te devuelva el libro con más páginas.
- Un método que te devuelva la primera estantería con más libros.
- Un método que te devuelva la primera estantería con más páginas.
- Cambia el main para que realice las siguientes acciones:
    - Primero consigue el acceso a la biblioteca.
    - Después te pide introducir todos los libros secuencialmente.
    - Después te pide eliminar 5 libros. Detecta si hay algún problema.
    - Después te pide introducir 5 libros en distintas estanterías. Detecta si hay algún problema.
    - Después te pide cambiar 4 libros de estantería. Detecta si hay algún problema.
    - Después te pide intercambiar libros 2 veces. Detecta si hay algún problema.
    - Muestra la biblioteca entera en formato csv.
    - Finalmente, muestra el libro con más páginas y en qué estantería se encuentra; muestra la estantería con más libros y también la estantería con más páginas.

### *Actividad 10: Pruebas unitarias con singleton*

Para ejecutar pruebas unitarias con singleton, debemos "trampear" el singleton. Una técnica sencilla es crear un método para restablecer la instancia, para así poder controlar de forma precisa qué datos tiene la instancia en cada momento. Este método es tan sencillo como lo que sigue:

```
public static void resetInstancia() {
        instancia = null;
    }
```

Con esto en mente, crea un conjunto de pruebas unitarias para todos los métodos de la clase `Biblioteca`.

---

## 3. Javadoc

Javadoc es una herramienta oficial de Java que se utiliza para generar documentación API en formato HTML directamente desde el código fuente. Esta documentación se basa en comentarios especiales que se escriben en el código, justo antes de las clases, métodos y otros elementos, y proporciona una manera fácil de crear y mantener documentación legible y coherente para los desarrolladores.

Javadoc utiliza comentarios específicos que comienzan con `/**` y terminan con `*/`, lo que los distingue de los comentarios regulares en Java (`//` y `/* ... */`). Los comentarios Javadoc incluyen etiquetas especiales:

- `@author`: Especifica quién escribió la clase.
- `@version`: Es la versión actual de la clase, útil para llevar control de cambios.
- `@since`: Indica la fecha o versión en la que la clase fue introducida o modificada.
- `@param` para describir parámetros.
- `@return` para describir el valor de retorno.
- `@throws` para excepciones (si las hay).

```java
/**
 * Clase que representa una cuenta bancaria simple con operaciones
 * básicas como depósito, retiro y consulta de saldo.
 *
 * Esta clase es utilizada para gestionar cuentas bancarias individuales
 * en una aplicación de banca.
 *
 *
 * @author Juan Pérez
 * @version 1.0
 * @since 2024-10-24
 */
public class CuentaBancaria {

    private double saldo;

    /**
     * Constructor para crear una cuenta bancaria con un saldo inicial.
     *
     * @param saldoInicial El saldo inicial de la cuenta.
     */
    public CuentaBancaria(double saldoInicial) {
        this.saldo = saldoInicial;
    }

    /**
     * Método para depositar dinero en la cuenta bancaria.
     *
     * @param monto El monto a depositar.
     */
    public void depositar(double monto) {
        this.saldo += monto;
    }
```

```java
    /**
     * Método para retirar dinero de la cuenta bancaria.
     *
     * @param monto El monto a retirar.
     * @throws IllegalArgumentException si el monto es mayor que el saldo.
     */
    public void retirar(double monto) {
        if (monto > saldo) {
            throw new IllegalArgumentException("Fondos insuficientes");
        }
        this.saldo -= monto;
    }

    /**
     * Método que devuelve el saldo actual de la cuenta.
     *
     * @return El saldo actual de la cuenta.
     */
    public double consultarSaldo() {
        return this.saldo;
    }
}
```

IntelliJ ofrece herramientas para generar tanto la documentación como la página web resultante de manera más o menos automatizada. Cuando colocas comentarios especiales de javadoc encima de un método, al pulsar intro después de `/**`, aparecerán los parámetros relevantes, así como el cierre del comentario. Una vez documentado el proyecto, seguimos los siguientes pasos en IntelliJ:

1. **Selecciona tu clase o paquete** en el explorador de proyectos, el que contiene el código que quieres documentar.
2. Ve a **Tools** → **Generate JavaDoc** (Herramientas → Generar JavaDoc).
3. Se abre una ventana donde puedes especificar opciones como:
   - El **directorio de salida** donde se guarda la documentación generada (generalmente un directorio como `docs` o `javadoc`).
   - Los **filtros** para elegir si quieres generar la documentación para todo el proyecto, solo algunas clases, etc.
   - La posibilidad de inclusión de **fuentes** o **archivos relacionados** en la documentación.
4. Haz clic en **OK**. IntelliJ ejecutará el proceso de generación de Javadoc.

Una vez completada la generación, se puede encontrar la documentación en el directorio especificado, normalmente en formato HTML. Busca el archivo `index.html`.

La documentación es especialmente importante cuando se trabaja en equipo o en proyectos grandes, pero es peligrosa si no se mantiene con regularidad, ya que los compiladores avisan

de errores en el código, pero normalmente no en la documentación. Puedes emplear modelos de generación de texto a través de Inteligencia Artificial, como *Chatgpt*, para generar documentación de tus clases, aunque el resultado no siempre será adecuado. Sin embargo, sí puede ayudarte a la hora de comprobar que la clase hace, efectivamente, lo que la documentación dice que hace la clase y corregir errores menores y de formato.

### *Actividad 11: Documenta el proyecto Biblioteca*

Crea un proyecto llamado Biblioteca si no lo tenías hecho ya. El proyecto contendrá un paquete también llamado `biblioteca` y dentro de él estarán las clases `Libro`, `Estanteria` y `Biblioteca` definidas en las actividades anteriores. Fuera del paquete, en el raíz del código fuente, estará la clase `Main`. Adapta el código para que se ejecute con normalidad. Incluye los test de la clase `Biblioteca`. Documéntalo todo en javadoc y genera la documentación. Con esto, ya tienes un proyecto sencillo de gestión de bibliotecas.

---

## 4. Diagramas en PlantUML y markdown

Estos apuntes están realizados en *VSCode*, empleando la extensión **"Markdown All in One"** y la extensión de *PlantUML* para generar los diagramas. Puedes consultar más sobre *PlantUML* en este enlace. Para que los diagramas se impriman bien al pasarlos a html o pdf, es necesario añadir en el archivo settings.json las siguientes líneas:

```
"markdown-pdf.plantumlOpenMarker": "```plantuml",
"markdown-pdf.plantumlCloseMarker": "```",
```

El archivo settings.json es accesible desde los comandos de *vscode*. Para entrar en ellos, pulsa F1 o escribe `>` dentro del buscador.

Al escribir diagramas se puede observar que la sintaxis no es muy diferente a hacerlos directamente sobre un lenguaje de programación convencional. Hay herramientas como Visual Paradigm que convierten el código en diagramas y también los diagramas en código.

### *Actividad 12: Diagramas en Markdown*

Consulta la documentación oficial de *PlantUML* y escribe todos los diagramas que has realizado hasta ahora en *vscode*. Puedes usar la extensión `.plantuml` y luego exportarlos como imagen, o bien integrarlos dentro de un archivo *markdown*.

---

# Programació Orientada a Objectes (VAL)

## 1. Mètodes i atributs estàtics

Existeix una paraula reservada que hem estat utilitzant fins ara sense saber molt bé què era: `static`. Els mètodes i atributs estàtics són aquells que no pertanyen a l'objecte, sinó a la classe.

Això vol dir que, una vegada el programa és executat, són sempre accessibles independentment de si hi ha o no instàncies de la classe que els conté.

En les primeres etapes del curs hem usat els elements estàtics per a simular el paradigma de programació procedural en Java, és a dir, la resolució de problemes mitjançant funcions/procediments i no mitjançant objectes.

Si declarem un atribut com a estàtic, es converteix en una espècie de variable amb àmbit dins de tota la classe i a la qual podem accedir des de fora depenent de la seua visibilitat. El mateix ocorre amb els mètodes.

Una cosa que cal tindre en compte és que no es poden utilitzar atributs de la classe dins de mètodes estàtics (perquè, en realitat, no té sentit ja que no hi ha instància de la classe), però sí que podem usar atributs estàtics dins de mètodes de l'objecte.

D'alguna manera, podem utilitzar les classes com a contenidors de mètodes estàtics, que es comporten com a llibreries de funcions. Tal és el cas de la classe `Math`, que inclou mètodes estàtics per a operacions matemàtiques complexes.

D'aquesta manera, podem emprar els atributs i mètodes estàtics com a ferramentes útils per a emprar dins o fora de la classe.

### *Activitat 8: Ús de mètodes i atributs estàtics*

Crea una variable estàtica dins de la classe llibre que servisca per a contar el nombre de llibres (instàncies) que hi ha. Emprala per a donar a cada llibre un identificador únic. Per a això, afegeix un atribut privat `id` de tipus enter. L'`id` no ha de passar-se com a atribut al constructor ni ha de poder ser modificat amb un setter.

Després, crea un mètode estàtic dins de la classe Prestatgeria que reba com a paràmetre una prestatgeria i torne un array de string. El contingut de l'array de string és cada un dels paràmetres, inclòs l'`id`, separat pel caràcter `;`. Pots rodejar cada paràmetre entre ` ` si dins d'algun dels camps hi haguera un `;`. Cada llibre ocupa un espai diferent de l'array. A aquest format de text se li coneix com a CSV (Comma Separated Values) i és molt usat en la codificació de dades.

---

## 2. El patró de disseny Singleton

Un dels usos més particulars dels atributs i mètodes estàtics és mitjançant el patró de disseny Singleton. Aquest patró garanteix que només hi haja una instància d'una classe fent que el constructor siga privat.

Per a que funcione, dins de la classe hem d'afegir un atribut privat i estàtic de la pròpia classe, així com un getter per a aquest atribut. En el getter, si la instància no està inicialitzada, es crida al constructor. Després, en qualsevol cas, es torna la instància.

D'aquesta forma, si no hi ha instància, se'n crea una i, si n'hi ha, obtens la que ja hi havia.

```
public class Singleton {
    // Instància privada i estàtica de la classe
    private static Singleton instancia;

    // Constructor privat per a evitar la instanciació des de fora
    private Singleton() { }

    // Mètode públic i estàtic que proporciona accés a l'única instància
    public static Singleton getInstancia() {
        if (instancia == null) {
            instancia = new Singleton(); // Crear instància si no existeix
        }
        return instancia; // Retornar la instància única
    }
}
```

Encara que el patró singleton és molt útil per a emmagatzemar dades globals i de configuració, no està exempt de problemes:

- **Dificultat per al testeig i el manteniment:** La naturalesa global de la instància pot complicar les proves unitàries, ja que introdueix un acoblament fort. L'acoblament ocorre quan un mòdul o classe és altament dependent d'un altre, és a dir, necessita conéixer detalls específics de la implementació de l'altre per a funcionar i, per tant, és més complex de mantindre.
- **Problemes de concurrència:** En entorns multihil, pot haver-hi problemes si no es maneja adequadament l'accés a la instància. Això se soluciona amb els mecanismes propis de la programació concurrent, però afegeixen complexitat a l'execució del programa.

## *Activitat 9: La classe biblioteca*

La classe `Biblioteca` conté deu prestatgeries, numerades de l'1 al 10. Afegeix aquest atribut a la classe `Prestatgeria`. Només existeix una biblioteca. La biblioteca permet fer les següents coses:

- Afegir llibres nous. S'afegeixen a la primera prestatgeria disponible. Torna fals si la biblioteca està plena.
- Afegir llibres nous a prestatgeries concretes. Torna fals si la prestatgeria no té espai.

- Eliminar un llibre. Se li passa el isbn del llibre i, si està, torna veritable i l'elimina. Si no està, torna fals.
- Canviar un llibre de prestatgeria. Se li passa el isbn del llibre i el nombre de la prestatgeria. Torna veritable si es pot canviar o si ja estiguera en la prestatgeria. Torna fals si la prestatgeria nova està plena o si el llibre no existeix en la biblioteca.
- Millora el mètode anterior amb un enum que indique el que ha passat en cada cas.
- Intercanvia dos llibres de prestatgeria. Rep com a paràmetre el isbn dels dos llibres i torna veritable si el canvi és possible i fals si no ho és (per exemple, perquè un llibre no existeix en la biblioteca).
- Un mètode que, donat el isbn o el títol d'un llibre, et torne la prestatgeria en la qual es troba.
- Un mètode que et torne el llibre amb més pàgines.
- Un mètode que et torne la primera prestatgeria amb més llibres.
- Un mètode que et torne la primera prestatgeria amb més pàgines.
- Canvia el main per a que realitze les següents accions:
    - Primer aconsegueix l'accés a la biblioteca.
    - Després et demana introduir tots els llibres seqüencialment.
    - Després et demana eliminar 5 llibres. Detecta si hi ha algun problema.
    - Després et demana introduir 5 llibres en distintes prestatgeries. Detecta si hi ha algun problema.
    - Després et demana canviar 4 llibres de prestatgeria. Detecta si hi ha algun problema.
    - Després et demana intercanviar llibres 2 vegades. Detecta si hi ha algun problema.
    - Mostra la biblioteca sencera en format csv.
    - Finalment, mostra el llibre amb més pàgines i en quina prestatgeria es troba; mostra la prestatgeria amb més llibres i també la prestatgeria amb més pàgines.

### *Activitat 10: Proves unitàries amb singleton*

Per a executar proves unitàries amb singleton, hem de "trampejar" el singleton. Una tècnica senzilla és crear un mètode per a restablir la instància, per a així poder controlar de forma precisa quines dades té la instància en cada moment. Aquest mètode és tan senzill com el següent:

```
public static void resetInstancia() {
        instancia = null;
    }
```

Amb això en ment, crea un conjunt de proves unitàries per a tots els mètodes de la classe `Biblioteca`.

---

Aquí tienes el texto traducido al valenciano:

---

# 3. Javadoc

Javadoc és una ferramenta oficial de Java que s'utilitza per a generar documentació API en format HTML directament des del codi font. Aquesta documentació es basa en comentaris especials que s'escriuen en el codi, just abans de les classes, mètodes i altres elements, i proporciona una manera fàcil de crear i mantindre documentació llegible i coherent per als desenvolupadors.

Javadoc utilitza comentaris específics que comencen amb `/**` i acaben amb `*/`, el que els distingeix dels comentaris regulars en Java (`//` i `/* ... */`). Els comentaris Javadoc inclouen etiquetes especials:

- `@author`: Especifica qui ha escrit la classe.
- `@version`: És la versió actual de la classe, útil per a portar control de canvis.
- `@since`: Indica la data o versió en què la classe va ser introduïda o modificada.
- `@param` per a descriure paràmetres.
- `@return` per a descriure el valor de retorn.
- `@throws` per a excepcions (si n'hi ha).

```java
/**
 * Classe que representa un compte bancari simple amb operacions
 * bàsiques com a dipòsit, retirada i consulta de saldo.
 *
 * Aquesta classe s'utilitza per a gestionar comptes bancaris individuals
 * en una aplicació bancària.
 *
 *
 * @author Juan Pérez
 * @version 1.0
 * @since 2024-10-24
 */
public class CuentaBancaria {

    private double saldo;

    /**
     * Constructor per a crear un compte bancari amb un saldo inicial.
     *
     * @param saldoInicial El saldo inicial del compte.
     */
    public CuentaBancaria(double saldoInicial) {
        this.saldo = saldoInicial;
    }

    /**
     * Mètode per a dipositar diners en el compte bancari.
     *
     * @param monto L'import a dipositar.
     */
    public void depositar(double monto) {
        this.saldo += monto;
    }
```

```
    /**
     * Mètode per a retirar diners del compte bancari.
     *
     * @param monto L'import a retirar.
     * @throws IllegalArgumentException si l'import és major que el saldo.
     */
    public void retirar(double monto) {
        if (monto > saldo) {
            throw new IllegalArgumentException("Fons insuficients");
        }
        this.saldo -= monto;
    }

    /**
     * Mètode que retorna el saldo actual del compte.
     *
     * @return El saldo actual del compte.
     */
    public double consultarSaldo() {
        return this.saldo;
    }
}
```

IntelliJ ofereix ferramentes per a generar tant la documentació com la pàgina web resultant de manera més o menys automatitzada. Quan col·loques comentaris especials de javadoc damunt d'un mètode, en polsar intro després de `/**`, apareixeran els paràmetres rellevants, així com el tancament del comentari. Una vegada documentat el projecte, seguim els passos següents en IntelliJ:

1. **Selecciona la teua classe o paquet** en l'explorador de projectes, el que conté el codi que vols documentar.
2. Ves a **Tools** → **Generate JavaDoc** (Eines → Generar JavaDoc).
3. S'obri una finestra on pots especificar opcions com:
   - El **directori de sortida** on es guarda la documentació generada (generalment un directori com `docs` o `javadoc`).
   - Els **filtres** per a triar si vols generar la documentació per a tot el projecte, només algunes classes, etc.
   - La possibilitat d'inclusió de **fonts** o **arxius relacionats** en la documentació.
4. Fes clic en **OK**. IntelliJ executarà el procés de generació de Javadoc.

Una vegada completada la generació, es pot trobar la documentació en el directori especificat, normalment en format HTML. Busca l'arxiu `index.html`.

La documentació és especialment important quan es treballa en equip o en projectes grans, però és perillosa si no es manté amb regularitat, ja que els compiladors avisen d'errors en el codi, però normalment no en la documentació. Pots emprar models de generació de text

mitjançant Intel·ligència Artificial, com *ChatGPT*, per a generar documentació de les teues classes, encara que el resultat no sempre serà adequat. No obstant això, sí que et pot ajudar a l'hora de comprovar que la classe fa, efectivament, el que la documentació diu que fa la classe i corregir errors menors i de format.

### *Activitat 11: Documenta el projecte Biblioteca*

Crea un projecte anomenat Biblioteca si no el tenies fet ja. El projecte contindrà un paquet també anomenat `biblioteca` i dins d'ell estaran les classes `Llibre`, `Prestatgeria` i `Biblioteca` definides en les activitats anteriors. Fora del paquet, en l'arrel del codi font, estarà la classe `Main`. Adapta el codi perquè s'execute amb normalitat. Inclou els tests de la classe `Biblioteca`. Documenta-ho tot en javadoc i genera la documentació. Amb això, ja tens un projecte senzill de gestió de biblioteques.

---

## 4. Diagrames en PlantUML i markdown

Aquests apunts estan realitzats en *VSCode*, emprant l'extensió **"Markdown All in One"** i l'extensió de *PlantUML* per a generar els diagrames. Pots consultar més sobre *PlantUML en aquest enllaç. Per a què els diagrames es mostren bé al passar-los a HTML o PDF, és necessari afegir en l'arxiu settings.json les línies següents:

```
"markdown-pdf.plantumlOpenMarker": "```plantuml",
"markdown-pdf.plantumlCloseMarker": "```",
```

L'arxiu settings.json és accessible des dels comandos de *vscode*. Per a entrar en ells, polsa F1 o escriu `>` dins del buscador.

En escriure diagrames es pot observar que la sintaxi no és molt diferent a fer-los directament sobre un llenguatge de programació convencional. Hi ha ferramentes com Visual Paradigm que converteixen el codi en diagrames i també els diagrames en codi.

### *Activitat 12: Diagrames en Markdown*

Consulta la documentació oficial de *PlantUML* i escriu tots els diagrames que has realitzat fins ara en *vscode*. Pots utilitzar l'extensió `.plantuml` i després exportar-los com a imatge, o bé integrar-los dins d'un arxiu *markdown*.

---